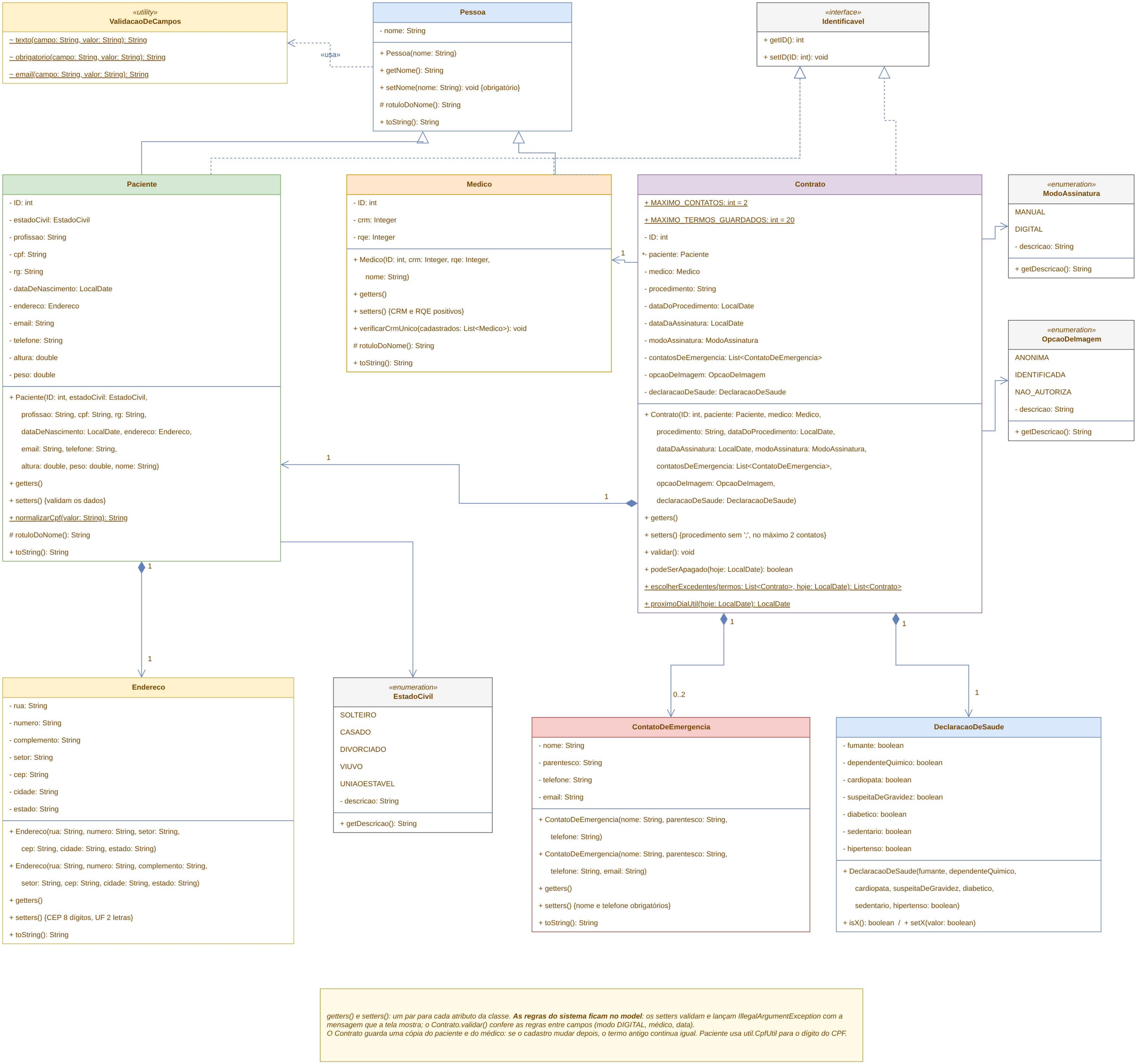
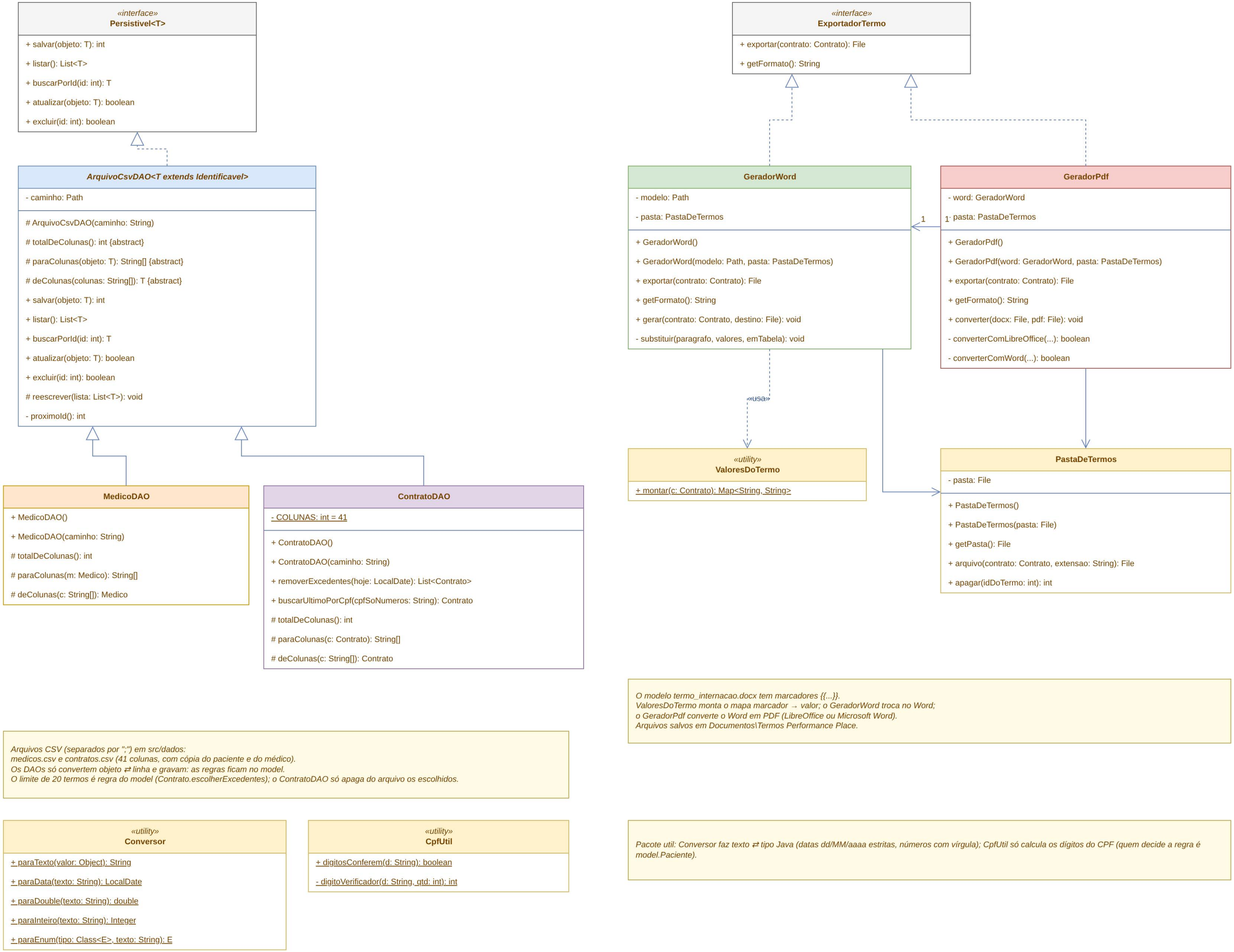

«utility»
ValidacaoDeCampos
~ texto(campo: String, valor: String): String
~ obrigatorio(campo: String, valor: String): String
~ email(campo: String, valor: String): String
«usa»
Pessoa
- nome: String
+ Pessoa(nome: String)
+ getNome(): String
+ setNome(nome: String): void {obrigatório}
rotuloDoNome(): String
+ toString(): String
«interface»
Identificavel
+ getID(): int
+ setID(ID: int): void
Paciente
- ID: int
- estadoCivil: EstadoCivil
- profissao: String
- cpf: String
- rg: String
- dataDeNascimento: LocalDate
- endereco: Endereco
- email: String
- telefone: String
- altura: double
- peso: double
+ Paciente(ID: int, estadoCivil: EstadoCivil,
profissao: String, cpf: String, rg: String,
dataDeNascimento: LocalDate, endereco: Endereco,
email: String, telefone: String,
altura: double, peso: double, nome: String)
+ getters()
+ setters() {validam os dados}
+ normalizarCpf(valor: String): String
rotuloDoNome(): String
+ toString(): String
Medico
- ID: int
- crm: Integer
- rqe: Integer
+ Medico(ID: int, crm: Integer, rqe: Integer,
nome: String)
+ getters()
+ setters() {CRM e RQE positivos}
+ verificarCrmUnico(cadastrados: List<Medico>): void
rotuloDoNome(): String
+ toString(): String
Contrato
+ MAXIMO_CONTATOS: int = 2
+ MAXIMO_TERMOS_GUARDADOS: int = 20
- ID: int
- paciente: Paciente
- medico: Medico
- procedimento: String
- dataDoProcedimento: LocalDate
- dataDaAssinatura: LocalDate
- modoAssinatura: ModoAssinatura
- contatosDeEmergencia: List<ContatoDeEmergencia>
- opcaoDeImagem: OpcaoDeImagem
- declaracaoDeSaude: DeclaracaoDeSaude
+ Contrato(ID: int, paciente: Paciente, medico: Medico,
procedimento: String, dataDoProcedimento: LocalDate,
dataDaAssinatura: LocalDate, modoAssinatura: ModoAssinatura,
contatosDeEmergencia: List<ContatoDeEmergencia>,
opcaoDeImagem: OpcaoDeImagem,
declaracaoDeSaude: DeclaracaoDeSaude)
+ getters()
+ setters() {procedimento sem ';', no máximo 2 contatos}
+ validar(): void
+ podeSerApagado(hoje: LocalDate): boolean
+ escolherExcedentes(termos: List<Contrato>, hoje: LocalDate): List<Contrato>
+ proximoDiaUtil(hoje: LocalDate): LocalDate
«enumeration»
ModoAssinatura
MANUAL
DIGITAL
- descricao: String
+ getDescricao(): String
«enumeration»
OpcaoDeImagem
ANONIMA
IDENTIFICADA
NAO_AUTORIZA
- descricao: String
+ getDescricao(): String
Endereco
- rua: String
- numero: String
- complemento: String
- setor: String
- cep: String
- cidade: String
- estado: String
+ Endereco(rua: String, numero: String, setor: String,
cep: String, cidade: String, estado: String)
+ Endereco(rua: String, numero: String, complemento: String,
setor: String, cep: String, cidade: String, estado: String)
+ getters()
+ setters() {CEP 8 dígitos, UF 2 letras}
+ toString(): String
«enumeration»
EstadoCivil
SOLTEIRO
CASADO
DIVORCIADO
VIUVO
UNIAOESTAVEL
- descricao: String
+ getDescricao(): String
ContatoDeEmergencia
- nome: String
- parentesco: String
- telefone: String
- email: String
+ ContatoDeEmergencia(nome: String, parentesco: String,
telefone: String)
+ ContatoDeEmergencia(nome: String, parentesco: String,
telefone: String, email: String)
+ getters()
+ setters() {nome e telefone obrigatórios}
+ toString(): String
DeclaracaoDeSaude
- fumante: boolean
- dependenteQuimico: boolean
- cardiopata: boolean
- suspeitaDeGravidez: boolean
- diabetico: boolean
- sedentario: boolean
- hipertenso: boolean
+ DeclaracaoDeSaude(fumante, dependenteQuimico,
cardiopata, suspeitaDeGravidez, diabetico,
sedentario, hipertenso: boolean)
+ isX(): boolean / + setX(valor: boolean)
1
1
1
1
1
1
1
1
1
0..2
1
getters() e setters(): um par para cada atributo da classe. As regras do sistema ficam no model: os setters validam e lançam IllegalArgumentException com a mensagem que a tela mostra; o Contrato.validar() confere as regras entre campos (modo DIGITAL, médico, data).
O Contrato guarda uma cópia do paciente e do médico: se o cadastro mudar depois, o termo antigo continua igual. Paciente usa util.CpfUtil para o dígito do CPF.


«interface»
Persistivel<T>
+ salvar(objeto: T): int
+ listar(): List<T>
+ buscarPorId(id: int): T
+ atualizar(objeto: T): boolean
+ excluir(id: int): boolean
ArquivoCsvDAO<T extends Identificavel>
- caminho: Path
ArquivoCsvDAO(caminho: String)
totalDeColunas(): int {abstract}
paraColunas(objeto: T): String[] {abstract}
deColunas(colunas: String[]): T {abstract}
+ salvar(objeto: T): int
+ listar(): List<T>
+ buscarPorId(id: int): T
+ atualizar(objeto: T): boolean
+ excluir(id: int): boolean
reescrever(lista: List<T>): void
- proximoId(): int
MedicoDAO
+ MedicoDAO()
+ MedicoDAO(caminho: String)
totalDeColunas(): int
paraColunas(m: Medico): String[]
deColunas(c: String[]): Medico
ContratoDAO
- COLUNAS: int = 41
+ ContratoDAO()
+ ContratoDAO(caminho: String)
+ removerExcedentes(hoje: LocalDate): List<Contrato>
+ buscarUltimoPorCpf(cpfSoNumeros: String): Contrato
totalDeColunas(): int
paraColunas(c: Contrato): String[]
deColunas(c: String[]): Contrato
«interface»
ExportadorTermo
+ exportar(contrato: Contrato): File
+ getFormato(): String
GeradorWord
- modelo: Path
- pasta: PastaDeTermos
+ GeradorWord()
+ GeradorWord(modelo: Path, pasta: PastaDeTermos)
+ exportar(contrato: Contrato): File
+ getFormato(): String
+ gerar(contrato: Contrato, destino: File): void
- substituir(paragrafo, valores, emTabela): void
GeradorPdf
- word: GeradorWord
- pasta: PastaDeTermos
+ GeradorPdf()
+ GeradorPdf(word: GeradorWord, pasta: PastaDeTermos)
+ exportar(contrato: Contrato): File
+ getFormato(): String
+ converter(docx: File, pdf: File): void
- converterComLibreOffice(...): boolean
- converterComWord(...): boolean
1
1
«usa»
«utility»
ValoresDoTermo
+ montar(c: Contrato): Map<String, String>
PastaDeTermos
- pasta: File
+ PastaDeTermos()
+ PastaDeTermos(pasta: File)
+ getPasta(): File
+ arquivo(contrato: Contrato, extensao: String): File
+ apagar(idDoTermo: int): int
O modelo termo_internacao.docx tem marcadores {{...}}.
ValoresDoTermo monta o mapa marcador → valor; o GeradorWord troca no Word;
o GeradorPdf converte o Word em PDF (LibreOffice ou Microsoft Word).
Arquivos salvos em Documentos\Termos Performance Place.
Arquivos CSV (separados por ";") em src/dados:
medicos.csv e contratos.csv (41 colunas, com cópia do paciente e do médico).
Os DAOs só convertem objeto ⇄ linha e gravam: as regras ficam no model.
O limite de 20 termos é regra do model (Contrato.escolherExcedentes); o ContratoDAO só apaga do arquivo os escolhidos.
«utility»
Conversor
+ paraTexto(valor: Object): String
+ paraData(texto: String): LocalDate
+ paraDouble(texto: String): double
+ paraInteiro(texto: String): Integer
+ paraEnum(tipo: Class<E>, texto: String): E
«utility»
CpfUtil
+ digitosConferem(d: String): boolean
- digitoVerificador(d: String, qtd: int): int
Pacote util: Conversor faz texto ⇄ tipo Java (datas dd/MM/aaaa estritas, números com vírgula); CpfUtil só calcula os dígitos do CPF (quem decide a regra é model.Paciente).

TelaMedico

- medicoController: MedicoController
- listaMedicos: List<MedicoDTO>

- montarMedico(): MedicoDTO
- carregarTabela(): void

TelaContrato

- contratoController: ContratoController
- medicoController: MedicoController
- listaMedicos: List<MedicoDTO>

- montarContrato(): ContratoDTO
- montarPaciente(): PacienteDTO
- carregarContrato(c: ContratoDTO): void
- buscarPorCpf(): void

TelaContratosSalvos

- contratoController: ContratoController
- todos: List<ContratoDTO>

+ carregar(): void
- selecionado(): ContratoDTO

«utility»
GeracaoDeDocumentos

+ gerarEMostrar(pai: Window, contrato: ContratoDTO): void

«usa»

MedicoController

- medicoDAO: Persistivel<Medico>

+ MedicoController()
+ MedicoController(medicoDAO: Persistivel<Medico>)
+ cadastrar(dto: MedicoDTO): int
+ listar(): List<MedicoDTO>
+ atualizar(dto: MedicoDTO): boolean
+ excluir(id: int): boolean
~ paraModel(dto: MedicoDTO): Medico
~ paraDTO(m: Medico): MedicoDTO

«cria»

«interface»
Persistivel<Medico>

(página 2: MedicoDAO)

«model»
Medico

(página 1)

«usa»

ContratoController

- contratoDAO: ContratoDAO
- pasta: PastaDeTermos
- ultimosRemovidos: List<Integer>

+ ContratoController()
+ ContratoController(dao: ContratoDAO, pasta: PastaDeTermos)
+ cadastrar(dto: ContratoDTO): int
+ atualizar(dto: ContratoDTO): boolean
+ excluir(id: int): boolean
+ listar(): List<ContratoDTO>
+ buscarPorId(id: int): ContratoDTO
+ buscarUltimoPorCpf(cpf: String): ContratoDTO
+ getUltimosRemovidos(): List<Integer>
+ getMaximoDeTermos(): int
+ sugerirData(): String
+ gerarWord(id: int): File
+ gerarDocumentos(id: int): File
+ arquivoWord(id: int): File
+ estadosCivis(): String[]
+ modosDeAssinatura(): String[]
+ opcoesDeImagem(): String[]
+ descricaoDoModo(nome: String): String
~ paraModel(dto: ContratoDTO): Contrato
~ paraDTO(c: Contrato): ContratoDTO

«usa»

«persistencia»
ContratoDAO

(página 2)

«documento»
PastaDeTermos

(página 2)

«cria»

«model»
Contrato

(página 1)

«documento»
GeradorWord / GeradorPdf

(página 2)

«usa»

«usa»

MedicoDTO

- id: int
- nome: String
- crm: String
- rqe: String

+ MedicoDTO()
+ MedicoDTO(id: int, nome: String, crm: String, rqe: String)
+ getters() / setters()

0..1 — *

ContratoDTO

- id: int
- paciente: PacienteDTO
- medico: MedicoDTO
- procedimento: String
- dataDoProcedimento: String
- dataDaAssinatura: String
- modoAssinatura: String
- opcaoDeImagem: String
- contatosDeEmergencia: List<ContatoDeEmergenciaDTO>
- declaracaoDeSaude: DeclaracaoDeSaudeDTO

+ ContratoDTO()
+ getters() / setters()

1 — 1

PacienteDTO

- nome: String
- cpf: String
- rg: String
- dataDeNascimento: String
- estadoCivil: String
- profissao: String
- altura: String
- peso: String
- telefone: String
- email: String
- endereco: EnderecoDTO

+ PacienteDTO()
+ getters() / setters()

1 — 1

EnderecoDTO

- rua: String
- numero: String
- complemento: String
- setor: String
- cep: String
- cidade: String
- estado: String

+ EnderecoDTO()
+ getters() / setters()

1 — 0..*

ContatoDeEmergenciaDTO

- nome: String
- parentesco: String
- telefone: String
- email: String

+ ContatoDeEmergenciaDTO()
+ ContatoDeEmergenciaDTO(nome, parentesco, telefone, email: String)
+ getters() / setters()

1 — 0..1

DeclaracaoDeSaudeDTO

- fumante: boolean
- dependenteQuimico: boolean
- cardiopata: boolean
- suspeitaDeGravidez: boolean
- diabetico: boolean
- sedentario: boolean
- hipertenso: boolean

+ DeclaracaoDeSaudeDTO()
+ isX() / setX(valor: boolean)

Padrão MVC + DTO
Visão (telas Swing) → Controller → Model (regras) → DAO (arquivo CSV).
As telas só conhecem DTO e controller. O DTO só transporta os dados, como texto, do jeito que foram digitados. O controller converte DTO ⇄ model e chama o DAO. O model valida e lança a mensagem de erro que a tela mostra.